

Front-end Essentials

jQuery & AJAX



Table of Contents

1. jQuery Overview
2. jQuery Syntax
3. jQuery Event
4. jQuery Callback Functions
5. AJAX

Lesson Objectives

- Understand the core concepts of jQuery
- Able to use jQuery to manipulate HTML
- Understand the core concepts of AJAX
- Able to use AJAX (thanks to jQuery) to create dynamic web page

Section 1

jQuery Overview

jQuery Overview – What is jQuery?

- jQuery is a **library** of **JavaScript** Functions.
- jQuery greatly **simplifies** JavaScript programming.
- jQuery is **easy** to learn.



- jQuery is a lightweight "write less, do more" JavaScript library.
- The purpose of jQuery is to make it much easier to use JavaScript on your website.



- **The jQuery library contains the following features:**
 - HTML element selections
 - HTML element manipulation
 - CSS manipulation
 - HTML event functions
 - JavaScript Effects and animations
 - HTML DOM traversal and modification
 - AJAX
 - Utilities

- The jQuery **library** is stored as a single JavaScript file, containing all the jQuery methods.
- It can be added to a web page with the following mark-up:

```
<head>  
<script type="text/javascript" src = "jquery.js"></script>  
</head>
```

- Or you can use the hosted jQuery library from **Google** or **Microsoft**.

```
<head>  
<script src="https://ajax.googleapis.com/ajax/libs/jquery  
/1.11.3/jquery.min.js"></script>  
</head>
```

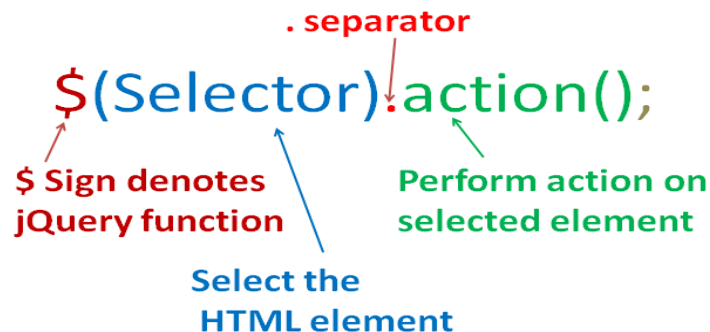
- A **library** of **JavaScript** Functions.
- A lightweight "**write less, do more**" JavaScript library.
- Contains many **features** that help us to developer web easier
- To Add jQuery: download its source code and add it to our web page (via **<script>** tag)

Section 2

jQuery Syntax

- The **jQuery syntax** is tailor made for **selecting** HTML elements and perform some **action** on the element(s).
- Basic syntax is:

`$(selector).action()`



The diagram illustrates the components of the jQuery syntax `$(selector).action();` with color-coded text and arrows:

- \$**: Red text with an arrow pointing to it, labeled "\$ Sign denotes jQuery function".
- (Selector)**: Blue text with an arrow pointing to it, labeled "Select the HTML element".
- .**: Red text with an arrow pointing to it, labeled ". separator".
- action()**: Green text with an arrow pointing to it, labeled "Perform action on selected element".

■ Examples:

- ✓ `$(this).hide()` - hides the current element.
- ✓ `$("p").hide()` - hides all `<p>` elements.
- ✓ `$(".test").hide()` - hides all elements with `class="test"`.
- ✓ `$("#test").hide()` - hides the element with `id="test"`.

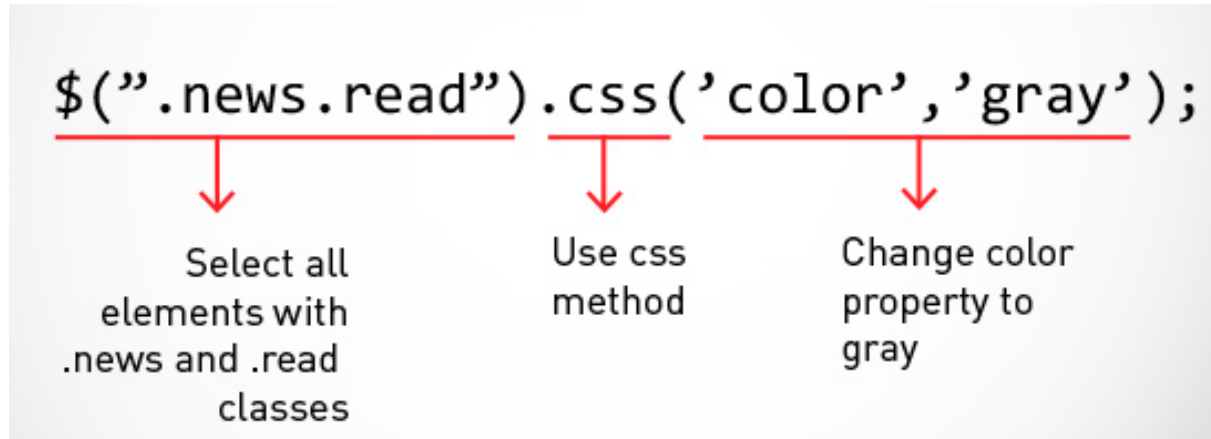
- You might have noticed that all jQuery methods **are inside** a document ready event:

```
$(document).ready(function(){  
    // jQuery methods go here...  
});
```

- This is to **prevent** any jQuery code from **running before** the document is **finished loading** (is ready).
 - ✓ It is good practice **to wait for the document to be fully loaded** and ready before working with it.

- Here are some examples of **actions** that can **fail** if methods are run before the document is fully loaded:
 - ✓ Trying to hide an element that is not created yet
 - ✓ Trying to get the size of an image that is not loaded yet

- jQuery selectors are one of the **most important parts** of the jQuery library.
- jQuery selectors allow you to **select** and then **manipulate** HTML element(s).



- jQuery selectors are used to "find" (or select) HTML elements based on their **id**, **classes**, **types**, **attributes**, **values** of attributes and much more
- All selectors in jQuery **start** with the **dollar sign** and **parentheses**: `$()`.

jQuery Syntax – Selectors Examples (1)

Syntax	Description
<code>\$("*")</code>	Selects all elements
<code>\$(this)</code>	Selects the current HTML element
<code>\$("p.intro")</code>	Selects all <code><p></code> elements with <code>class="intro"</code>
<code>\$("p:first")</code>	Selects the first <code><p></code> element
<code>\$("ul li:first")</code>	Selects the first <code></code> element of the first <code></code>
<code>\$("ul li:first-child")</code>	Selects the first <code></code> element of every <code></code>

jQuery Syntax – Selectors Examples (2)

Syntax	Description
<code>\$("[href]")</code>	Selects all elements with an href attribute
<code>\$("a[target='_blank']")</code>	Selects all <a> elements with a target attribute value equal to "_blank"
<code>\$("a[target!='_blank']")</code>	Selects all <a> elements with a target attribute value NOT equal to "_blank"
<code>\$(":button")</code>	Selects all <button> elements and <input> elements of type="button"
<code>\$("tr:even")</code>	Selects all even <tr> elements
<code>\$("tr:odd")</code>	Selects all odd <tr> elements

- The jQuery **element selector** selects elements based on the **element name**.
- Example 1:
`$("p")`
- Example 2:

```
$(document).ready(function(){  
    $("button").click(function(){  
        $("p").hide();  
    });  
});
```
- Refs:

- Question 1: Use the correct selector to hide all elements with an href attribute.

```
<script>  
$(document).ready(function(){  
    $("selector").hide();  
});  
</script>
```



- The **jQuery syntax** is tailor made for **selecting** HTML elements and perform some **action** on the element(s).
- Basic syntax is:
\$(selector).action()
- Document Ready is to **prevent** any JavaScript/jQuery code from **running before** the document is **finished loading** (is ready).

Section 3

jQuery Events

- All the different **visitor's actions** that a web page can respond to are **called events**.
- **Examples:**
 - ✓ moving a mouse over an element
 - ✓ selecting a radio button
 - ✓ clicking on an element

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

jQuery Events – Example

```
<!DOCTYPE html>
<html>
<head>
<script type="text/javascript" src="jquery.js"></script>
<script type="text/javascript">
    $(document).ready(function() {
        $("button").click(function() {
            $("p").hide();
        });
    });
</script>
</head>
<body>
    <h2>This is a heading</h2>
    <p>This is a paragraph.</p>
    <p>This is another paragraph.</p>
    <button>Click me</button>
</body>
</html>
```

- **Events:** all the different **visitor's actions** that a web page can respond to
- **Common Events:** mouse click, mouse hover, keyup, keypress, document load, document unload

Section 4

jQuery Callback Functions

- A callback function is executed after the current effect is 100% finished.
- JavaScript statements are executed line by line. However, with effect, the next line of code can be run even though the animation is not finished. This can create errors.
 - ✓ To prevent this, you can create a callback function.
- A callback function is executed after the current effect is finished.

jQuery Callback Functions – Example

```
<!DOCTYPE html>
<html>
<head>
<script type="text/javascript" src="jquery.js"></script>
<script type="text/javascript">
$(document).ready(function() {
    $("button").click(function() {
        $("p").hide(1000, function() {
            alert("The paragraph is now hidden");
        });
    });
});
</script>
</head>
<body>
<button>Hide</button>
<p>This is a paragraph with little content.</p>
</body>
</html>
```

- Demo with:
 - ✓ **draggable** action;
 - ✓ **addClass** action;
 - ✓ **removeClass** action;

Code snippet

```
$(function() {  
    $("#button").on("click", function() {  
        $("#effect").addClass("newClass");  
    });  
  
    function callback() {  
        setTimeout(function() {  
            $("#effect").removeClass("newClass");  
        }, 1500);  
    }  
});
```

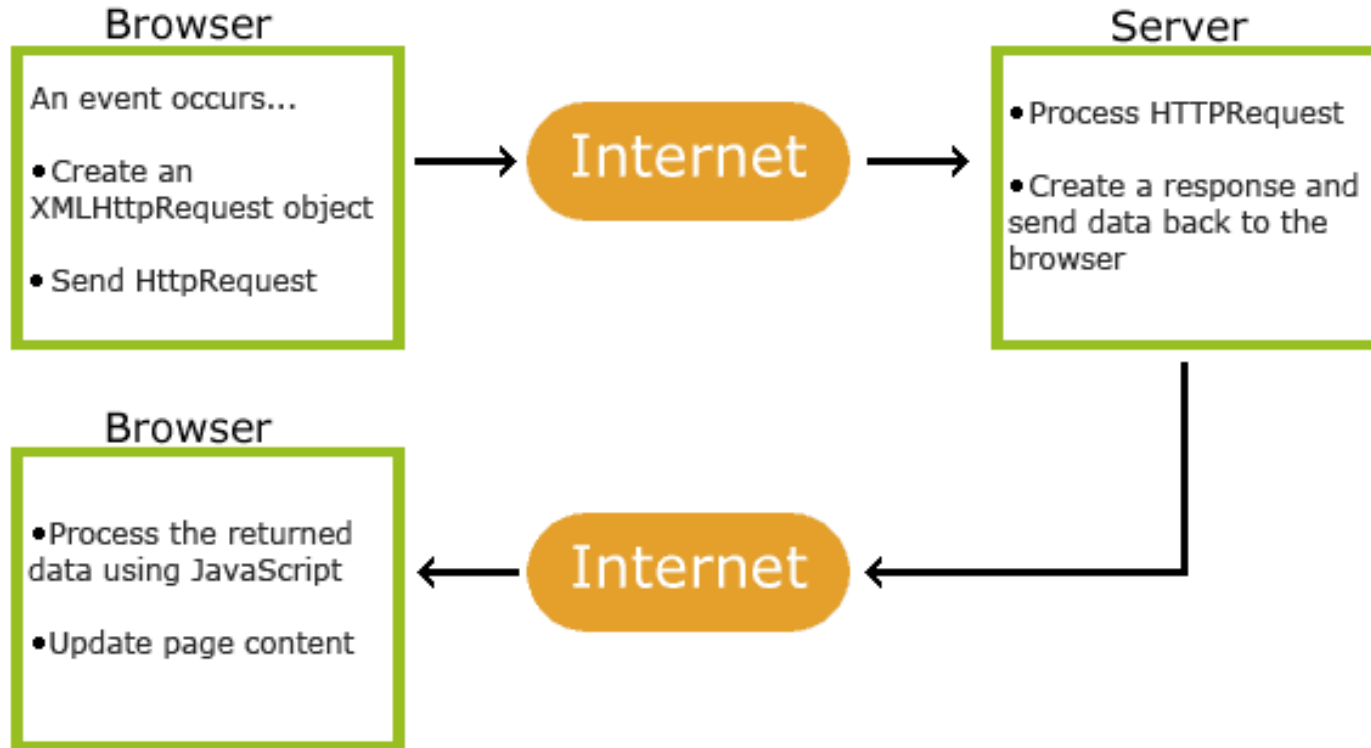
- A callback function is executed after the current effect is 100% finished.
- JavaScript statements are executed line by line.
- However, with effect, the next line of code can be run even though the animation is not finished.
- A callback function is executed after the current effect is finished.

Section 5

AJAX

- Asynchronous JavaScript and XML
- AJAX is the art of **exchanging data with a server**, and updating **parts of a web page** - without reloading the whole page.
- AJAX allows web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.

AJAX – How it works?



- AJAX is based on internet standards, and uses a combination of:
 - ✓ XMLHttpRequest object (to exchange data asynchronously with a server)
 - ✓ JavaScript/DOM (to display/interact with the information)
 - ✓ CSS (to style the data)
 - ✓ XML (often used as the format for transferring data)
- AJAX applications are browser- and platform-independent!

Code snippet

```
function loadXMLDoc() {  
    var xmlhttp;  
    if (window.XMLHttpRequest) { // code for IE7+, Firefox, Chrome, Opera, Safari  
        xmlhttp = new XMLHttpRequest();  
    } else { // code for IE6, IE5  
        xmlhttp = new ActiveXObject("Microsoft.XMLHTTP");  
    }  
    xmlhttp.onreadystatechange = function() {  
        if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {  
            document.getElementById("myDiv").innerHTML=xmlhttp.responseText;  
        }  
    }  
    xmlhttp.open("GET", "ajax_info.txt", true);  
    xmlhttp.send();  
}
```

- jQuery provides a rich set of methods for AJAX web development.
- With jQuery AJAX, you can request TXT, HTML, XML or JSON data from a remote server using both HTTP Get and HTTP Post
- **And you can load remote data directly into selected HTML elements of your web page!**

- The jQuery `load()` method is a simple (but very powerful) AJAX function. It has the following syntax:

`$(selector).load(url,data,callback)`

- Use the **selector** to define the HTML element(s) to change, and the **url parameter** to specify a web address for your data.

■ Syntax:

```
jQuery.ajax({  
    type:"POST",//Phương thức gửi request là POST hoặc GET  
    data:"id=12&name=abc", //tham số gửi kèm  
    dataType:"xml",//kiểu dữ liệu trả về, mặc định là text  
    url:"/login/servletLogin",//Đường dẫn tới nơi xử lý request ajax  
    success: function (){//hàm gọi về khi thực hiện thành công  
        // mã lệnh  
    }  
});
```

AJAX – Example (1)

```
<script type="text/javascript">
    jQuery(document).ready(function() {
        jQuery("#ajaxButton").click(function() {
            jQuery.ajax({
                type : "POST",
                url : "ajax.html",
                success : function(html) {
                    jQuery("#responseDiv").html(html);
                }
            });
        });
    });
</script>
```

AJAX – Example (2)

```
<style>
#sampleTable {
  border-collapse: collapse;
}
#sampleTable td {
  border: 1px solid black;
  width: 100px;
}
</style>
<table id="sampleTable">
  <tr>
    <td>Name</td>
    <td>Year</td>
  </tr>
  <tr>
    <td>Van A</td>
    <td>1982</td>
  </tr>
</table>
```

- AJAX is the art of **exchanging data with a server**, and updating **parts of a web page** - without reloading the whole page.
- AJAX allows web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes
- jQuery provides a rich set of methods for AJAX web development: `load()` method and `ajax()` method

Thank you

